Secure Coding with LLMs - Title TBD

1st Kaiqi Liang
CSSE Department
University of Western Australia
Perth, WA
23344153@student.uwa.edu.au

2nd Corey Saxon
CSSE Department
University of Western Australia
Perth, WA
23178835@student.uwa.edu.au

3rd Michael Hodgson
CSSE Department
University of Western Australia
Perth, WA
email address or ORCID

4th Woojin Jeon
Dept. of Electrical and Computer Engineering
Sungkyunkwan University
Suwon, Korea
dnwls0116@naver.com

5th Larry Huynh
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Jin Hong
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Despite its widespread use in industry and academia, the security of code generated by large language models (LLMs) has not had much research done towards it. Our group aims to fill this knowledge gap through the application of static analysis tools and integration with the GPT4 API. By performing an iterative process of analysis and improvement, our pipeline can identify vulnerabilities in provided code and suggest prompts which may assist a user in constructing a prompt to initiate a response of secure code from GPT. Prompt engineering techniques are leveraged to best instruct the model how to perform these tasks and act in a predictable manner.**

[Corey: DO THE ABSTRACT LAST]

***Index Terms*—prompt engineering, static analysis, large-language model, secure code generation**

I. Introduction

LLMs have been making waves in many fields since their introduction. Their prevalence and ubiquity are undeniable, but their use is considered hazardous [Corey: DEFINITELY reword this. My brain is failing me] due to their own non-deterministic nature [Corey: I think we have a source to support this, if not I'll find one.]. Due to their popularity and inherent danger, the accuracy of their output has need to be assessed. Focusing on the field of secure coding, our aim is to investigate how often GPT provides codes with security vulnerabilities and how different kinds of prompting techniques change the presence of such vulnerabilities. This is done with the desirable outcome of providing a tool which not only improves the security of code output by GPT, but also assist the user in their prompt-writing to assure that future queries are more likely to result in secure code.

[Larry: state the problem and briefly describe how we propse to fix. this is where you inteoduxe the context. something like with LLMs, it may be possible to produce secure code; we want to investigate how. we provide pupepline for secure code generatoon with ... then give breif statement about outcome of the project]

II. Related Works

Due to the fast-paced evolution of OpenAI and it's products, there are little to no papers exploring the newer models of GPT and its application to secure coding. There are some which experiment with GPT3 and GPT3.5, and these are considered relevant enough to be used as primary sources.

There exists some research which explore the likelihood of vulnerability introduction in GPT-generated code [1], however these are often based on highly specialised pieces of code, with little programmatic scope and typically based in a single language. For the few which do consider greater variety in code and language, they do not consider how prompts can be engineered to improve code quality and also do not consider how GPT can be used to iteratively improve the code [2]. Other research looks into the generation of code resistant to hardware vulnerabilities [3] [4]. This is adjacent to the domain of our research but we focus on software vulnerabilities such as the CWE Top 25. A futher aspect of our research is concerned with prompt engineering and best practices. Researchers have worked on training a model to generate code and experimented with prompt design for that regard, but do not focus on code security in a meaningful way and neglect to consider the ease of model oversight [5] [6]. More common is research concerned with the application of GPT as a bug-fixing tool. It is typically agreed upon that ChatGPT is capable of outperforming traditional bug-fixing tools [7] [8] [9]. These papers fail to consider the generation of code by an LLM, and also rely on some manner of manual code review within their system. We hope to avoid this mandatory human involvement as much as possible by introducing GPT as a generator and static analysis tool, performing an iterative improvement

[Larry: try to make more generalized statements about what people did, rather than assessing each one individually. so something like "few previous works have attempted to generate/assess secure code via llms cite(1,2,3). while they have seen success, these works fail to consider (x, y, z), or don't fulfil some stan-

dard/requirement. In addition, these works provide single-shot, standalone assessments of security vulnerabilities; we believe these ideas can be enhanced through an iterative improvement model (briefly describe). We are aware of such methods being used in general/in other areas (cite works that have done similar), however (whatever sets us apart, maybe talk about static analysis tools, etc.)]

This is where we aim to abridge previous works by considering generated code and then applying some of these earlier attempted practices.

III. Pipeline

1

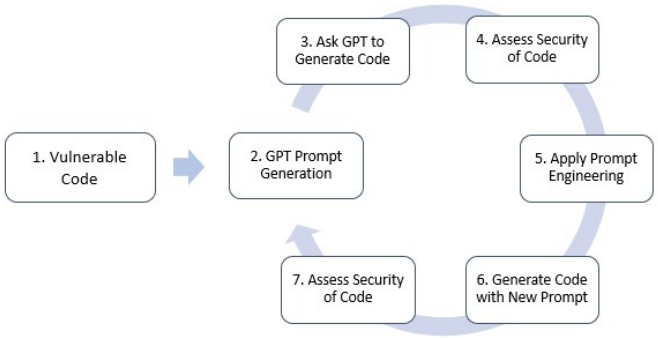

Fig. 1: The major steps of the system pipeline

[Larry: add figure for actual pipeline implementation. think design drawings and flowcharts]

A. Step 1: Vulnerable Code

Vulnerable code snippets, which are fed to the LLM, are sourced from the 2023 CWE Top 25 Most Dangerous Software Weaknesses. Most weaknesses include vulnerable code snippets for several languages under the Demonstrative Examples section. [Larry: change the remaining part to just something like "we also account for variety of most popular programming languages. then also make the observation that certain vulnerabilities and analysis tools are language specific, and give examples. then just say our model is robust to this.] Different languages were considered for their unique vulnerabilities but also to assess the scope of application of this pipeline. Currently, code built in C/C++, Python, Java, JavaScript and TypeScript are run over chosen static analyzers, where other languages will skip the step of static analysis.

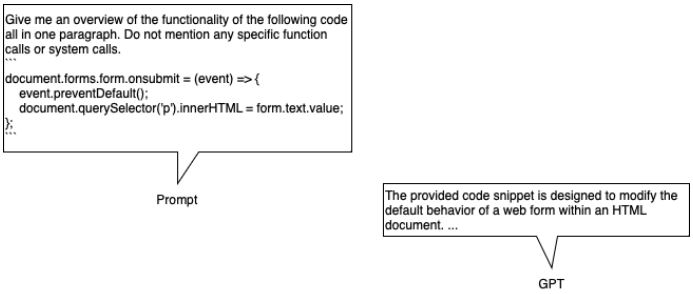

Fig. 2: Prompt for Step 2 of the Pipeline

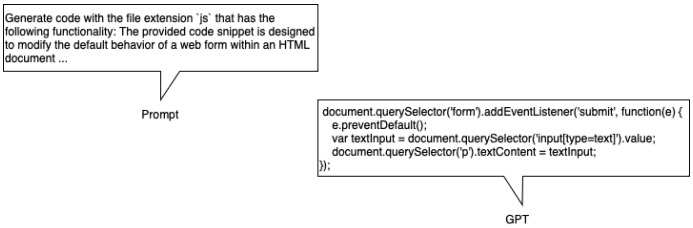

Fig. 3: Prompt for Step 3 of the Pipeline

B. Step 2: GPT Prompt Generation

In this step we ask GPT to generate a prompt given the vulnerable code which can be used in the next step to generate code. Given the vulnerable code, we initially form a prompt that requests GPT to write another prompt, which once given back to GPT should generate a version of the original code with the vulnerability removed. Shown in Figure 2 is an example of a prompt used for the XSS vulnerability.

C. Step 3: Code Generation

Next we used the prompt generated in the previous step and asked GPT to give us code described by the prompt. An example is shown in Figure 3.

`extension` makes sure that the code GPT generates next is in the same programming langauge as the vulnerable code.

D. Step 4: Security Assessment

Here is where our static analysis tools come in. Once the testable code has been generated, it must be assessed by some metric to measure how vulnerable it is. Several tools were considered but most were ultimately deemed unusable in our pipeline. This was due to either their inability to identify simple vulnerabilities shown to them or their lack of command line control options. If these tools worked solely as IDE plug-ins, they would not be suitable for an automated pipeline. These tools must also possess several properies which would make their use suitable. Their result format must be mostly *deterministic*, so that results are reproducible and to prevent divergent loops where a previously identified vulnerability is no longer found, despite it not being fixed. The tools must also have an expected output so that they may be parseable by the pipeline, and must have an acceptable degree of abstraction so that while training and developing the system we can identify any potential shortcomings.

[Corey: Considering adding a table depicting tools considered and why they were/werent accepted (tool, ability to find simple vuln, terminal control, accepted)]

1) SonarQube: While SonarQube is a widely used open-source platform for analyzing and tracking the quality of code [10] but it is a very complex software with a user interface which is difficult to integrate with a command line tool. Hence we opt out of that.

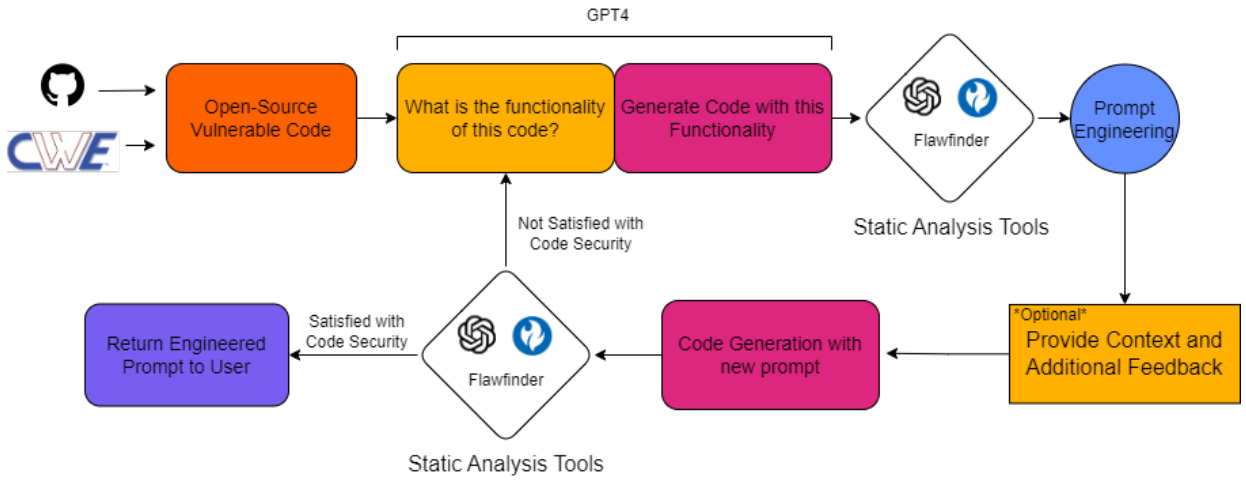

Fig. 4: How some vulnerable code may be processed by the pipeline

2) HCL AppScan: Great as an IDE plugin but hard to build into a pipeline run on the command line.

3) Flawfinder: Easy to set up, sometimes perform as well as the other open source tools [11].

4) SpotBugs: Better than PMD for Java [12].

5) Bearer CLI: Static Application Security Testing (SAST) to discover, filter and prioritize security and privacy risks using sensitive data flow analysis. Currently supports Java, Ruby, JavaScript and TypeScript.

6) Bandit: Bandit is a comprehensive source vulnerability scanner for Python.

E. Step 5: Prompt Engineering

Prompt engineering is the refinement of the prompts fed to LLMs to direct and manipulate the model to create predictable output. There are several techniques which assisted in our prompt engineering practices:

1) Clarity and Specificity
2) Context and Background Information
3) Tone and Style
4) Brevity and Focus
5) Goal Orientation [13]

An effective prompt should convey exactly what is expected to the LLM to provide relevant output. Contextual cues aid the model in understanding the role the code will play in the system and thus adjust its responses accordingly. The tone of the prompt used can influence the manner in which the model gives it outputs. While providing context is important, overly long prompts can confuse the model and dilute the users original intent. It is also important to have a clear goal in the prompt for the model to properly respond to.

F. Step 6: Generation with new Prompt

Using this engineered prompt, we generate new code which should now be free of some or all of the identified vulnerabilities.

G. Step 7: Reassessment

The earlier static analysis techniques are repeated to gauge if the number and severity of vulnerabilities have been reduced. If they have not, the pipeline will iterate again. If the model is satisfied with the improvement of the code, it will return the code along with the prompt that generated that code.

IV. Case Study

A. Static Analysis Tools

The traditional static analysis tools had a high rate of false negatives, they were not able to pick up several of the simple example vulnerabilities. The conducted experiments are shown in Table I. These results were particularly troubling due to the importance of being able to accurately identify where vulnerabilities exist in code in order to fix them.

B. Static Analysis Using GPT

Due to static analysis tools' insufficiency for finding vulnerabilities the decision was made to experiment with using GPT4. It was able to identify every single example vulnerability that we tested under the 2023 CWE Top 25 Most Dangerous Software Weaknesses. Upon further literature review and with papers supporting GPT's ability as an analysis tool [7] [9], GPT was introduced to the pipeline with the additional role as a static analysis tool.

C. Difficult Vulnerability

In the two case studies above the vulnerabilities we tested are very stripped down versions whose complexity is not likely to be representative of any real world software. We attempted to see if GPT can identify CVE-2017-9430, passing the entire dnstracer.c file to GPT4 exceeded the maximum number of tokens the current state of the model can handle. However we opted to give GPT only the main function and it found the intended Buffer Overflow vulnerability as shown in Figure 5.

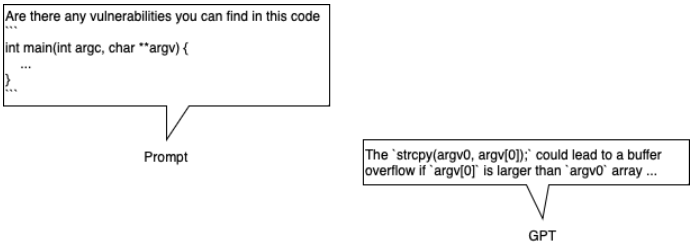

Fig. 5: Prompt for static analysing dnstracer.c

D. Iteration

[Corey: We need to properly define what metric we are using to limit our iteration. Is it going until there are no more vulnerabilities, but also with a cap? Is it doing a percentage reduction?]

V. Testing

[Corey: We should do testing of GPT as a static analysis tool, feeding it vulnerable code and measuring how often it finds the vulnerabilities. This will give us a useful figure but also provide some much-needed authenticity.]

VI. Code Improvement

[Corey: How often is the code improved? By how much? After how many iterations? Should consider building a table depicting languages used, CWEs used, and the actual improvement.]

TABLE I: List of CWE Vulnerabilities and the Detection Results Using Static Analysis Tools

CWE Vulnerability	**CWE ID**	**Language**	**Static Analysis Tool**	**Detected**
Out-of-bounds Write	CWE-787	C	Flawfinder	No
XSS	CWE-79	HTML	HCL AppScan	Yes
SQLi	CWE-89	JavaScript	Bearer	No
Use After Free	CWE-416	C	Flawfinder	No
OS Command Injection	CWE-78	C	Flawfinder	Yes
Improper Input Validation	CWE-20	C	Flawfinder	No
Out-of-bounds Read	CWE-125	Python	Bandit	No

VII. Conclusion

References

[1] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at c: A user study on the security implications of large language model code assistants," 2023.

[2] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by chatgpt?" 2023.

[3] M. Nair, R. Sadhukhan, and D. Mukhopadhyay, "Generating secure hardware using chatgpt resistant to cwes," Cryptology ePrint Archive, Paper 2023/212, 2023, https://eprint.iacr.org/2023/212. [Online]. Available: https://eprint.iacr.org/2023/212

[4] D. Saha, S. Tarek, K. Yahyaei, S. K. Saha, J. Zhou, M. Tehranipoor, and F. Farahmandi, "Llm for soc security: A paradigm shift," 2023.

[5] C. Liu, X. Bao, H. Zhang, N. Zhang, H. Hu, X. Zhang, and M. Yan, "Improving chatgpt prompt for code generation," 2023.

[6] M. L. Siddiq, B. Casey, and J. C. S. Santos, "A lightweight framework for high-quality code generation," 2023.

[7] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of chatgpt," 2023.

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.

[9] J. A. Prenner, H. Babii, and R. Robbes, "Can openai's codex fix bugs?: An evaluation on quixbugs," pp. 69–75, 2022.

[10] D. Marcilio, R. Bonifácio, E. Monteiro, E. Canedo, W. Luz, and G. Pinto, "Are static analysis violations really fixed? a closer look at realistic usage of sonarqube," in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, 2019, pp. 209–219.

[11] I. Bitdefender, "A comparison of static analysis tools for vulnerability detection in c/c++ code."

[12] A. Kaur and R. Nayyar, "A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code," *Procedia Computer Science*, vol. 171, pp. 2023–2029, 2020, third International Conference on Computing and Network Communications (CoCoNet'19). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050920312023

[13] S. Heyer, "Generative ai - best practices for llm prompt engineering." [Online]. Available: https://medium.com/google-cloud/generative-ai-best-practices-for-llm-prompt-engineering-2a0131c805cc.